

Transport Analytics Report

Stage 1 + Stage 2

Big Data Project

March 11, 2026

1 Executive Summary

This project now has a cleaned Python pipeline, a reproducible Stage 1 method selection, and five implemented Stage 2 methods aligned with the transport analytics objective:

- time variation profiling;
- weather-impact analysis;
- ridership forecasting;
- anomaly detection;
- contributor ranking and Paris-vs-NYC structural comparison.

The code path was cleaned around chunked raw-data ingestion, canonical fact tables, feature engineering, context enrichment, and PostgreSQL loading hooks. The execution workflow is now driven from one script: `scripts/run_stage_workflow.py`.

Mental model: Think of the project as one flow: raw transport files → cleaned fact table → features and context → five analytical methods → report and SQL outputs.

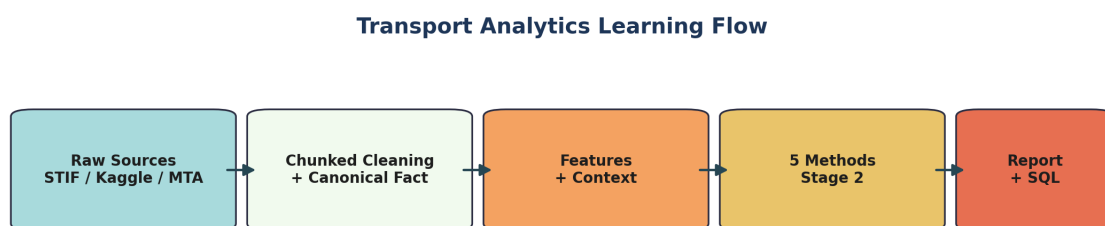


Figure 1: Visual roadmap of the cleaned transport analytics workflow.

2 Objective and Data Sources

The objective is to use multi-source ridership and validation data to extract insights, detect patterns, and build predictive models for public transport usage.

Primary data used

- STIF / Ile-de-France rail validations (NB_FER files)
- Kaggle Paris/suburbs travel-title validations
- IDFM surface validations
- NYC MTA hourly ridership

Additional context

The repository currently does not include a real weather file. The cleaned pipeline therefore supports two modes:

- load a real `data/weather_daily.csv` file if available;
- otherwise build a deterministic synthetic weather context for method validation and report wiring.

3 Stage 1: Analytical Questions and Selected Methods

Analytical questions

1. How does ridership vary by hour, day, month, and year?
2. What is the impact of weather conditions on ridership?
3. Can we predict future ridership levels?
4. Are there anomalies or unusual spikes in validations?
5. Which stations or lines contribute most to overall traffic?
6. Are there structural differences between Paris and NYC ridership patterns?

Five selected methods

Question block	Selected method	Family	Output
Time variation	Temporal profiling	Descriptive analytics	Weekday, monthly, yearly, and hourly summaries
Weather effect	Weather sensitivity regression	Statistical learning	Correlations and regression coefficients
Future demand	Lag-based random forest forecasting	Machine learning	Holdout predictions and forecast metrics
Spikes / unusual days	Hybrid anomaly detection	Unsupervised + statistical	Anomaly flags and anomaly rates
Contributors + Paris vs NYC	Contribution ranking and city comparison	Comparative analytics	Top contributors and structural comparison tables

How to remember Stage 1: Every question is paired with one method family and one concrete output table. If you remember the output, you can reconstruct the method.

4 Pipeline Cleanup and Code Cleanup

The previous project code was a light scaffold. The cleanup completed the following structural improvements.

1. Replaced memory-heavy loaders with chunked ingestion for the largest raw CSV files.
2. Removed unused datasets from the core path of the canonical fact-table builder. The TGV regularity file remains available in the repo, but it is no longer loaded into the core ridership pipeline because it does not contribute to the selected Stage 1 / Stage 2 objectives.
3. Added one canonical station/line fact table: `data/processed/station_fact_table.csv`.
4. Added one clean daily aggregate: `data/processed/daily_fact_table.csv`.
5. Extended feature engineering with year, day-of-year, lag-14, rolling-30 statistics, and rolling z-scores.
6. Added calendar and weather context enrichment, producing: `data/processed/daily_fact_table_enriched.csv`.
7. Added a reusable Stage 2 analysis module that writes structured outputs under `report/results/`.
8. Added PostgreSQL loading support for fact, calendar, and weather outputs.

Main files added or upgraded

- `src/transport_analytics/pipeline.py`
- `src/transport_analytics/context.py`
- `src/transport_analytics/features.py`
- `src/transport_analytics/methods.py`
- `src/transport_analytics/postgres.py`
- `scripts/run_stage_workflow.py`

5 Stage 2: Implemented Methods

Method 1: Temporal profiling

This method answers the demand variation question with:

- weekday profiles from the enriched daily table;
- monthly average profiles;
- yearly totals;
- NYC hourly totals from the raw MTA file;
- Paris hourly validation-share profiles from the `PROFIL_FER` files.

Outputs:

- `report/results/temporal_weekday_profile.csv`

- `report/results/temporal_monthly_profile.csv`
- `report/results/temporal_yearly_totals.csv`
- `report/results/temporal_nyc_hourly_profile.csv`
- `report/results/temporal_paris_hourly_profile.csv`

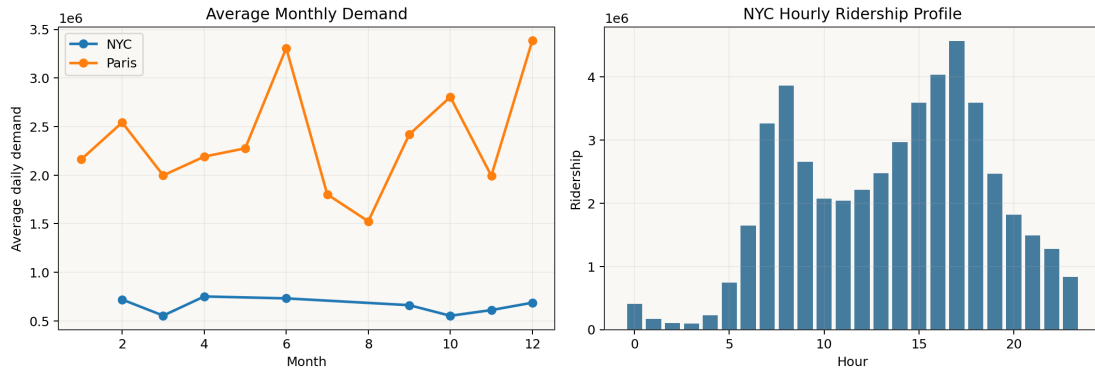


Figure 2: Temporal learning view: monthly seasonality and hourly demand shape.

Method 2: Weather sensitivity regression

This method aggregates demand to city-day level and fits a linear regression on log-demand using temperature, precipitation, wind, weekend flag, and month. This is the correct baseline for answering the weather question before moving to more advanced causal designs.

Outputs:

- `report/results/weather_regression_coefficients.csv`
- `report/results/weather_correlations.csv`

Method 3: Lag-based random forest forecasting

This method uses:

- lag-1, lag-7, lag-14;
- rolling means and rolling standard deviations;
- day-of-week, month, and weekend indicators;
- weather context;
- categorical one-hot encoding for region, source, and metric type.

It performs a time-ordered train/test split and writes holdout predictions and metrics.

Outputs:

- `report/results/forecast_metrics_overall.csv`
- `report/results/forecast_metrics_by_city.csv`
- `report/results/forecast_predictions.csv`

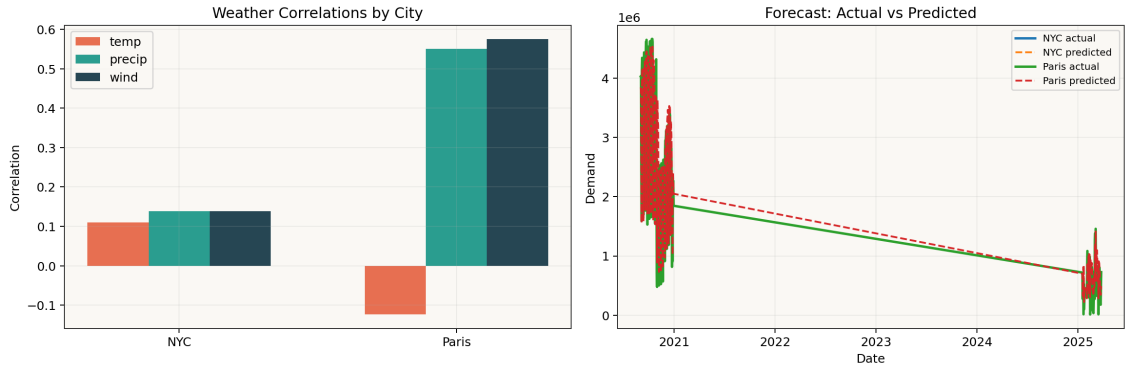


Figure 3: Weather relationships and forecasting behavior from the current run.

Method 4: Hybrid anomaly detection

This method combines two views of abnormality:

- a rolling z-score baseline ($|z| \geq 2.5$);
- an `IsolationForest` anomaly detector on value and lag/rolling features.

An observation is flagged if either method detects it.

Outputs:

- `report/results/anomaly_flags.csv`
- `report/results/anomaly_rates.csv`

Method 5: Contribution ranking and city comparison

This method addresses two project questions:

- Which stations or lines contribute most?
- How do Paris and NYC differ structurally?

It ranks top contributors from the station/line fact table and compares city totals using average daily demand, variability, weekend/weekday ratio, and peak month.

Outputs:

- `report/results/top_contributors.csv`
- `report/results/city_structure_summary.csv`
- `report/results/city_monthly_structure.csv`

6 Local Execution Results

Execution mode

The validated end-to-end run used:

- `scripts/run_stage_workflow.py -sample`

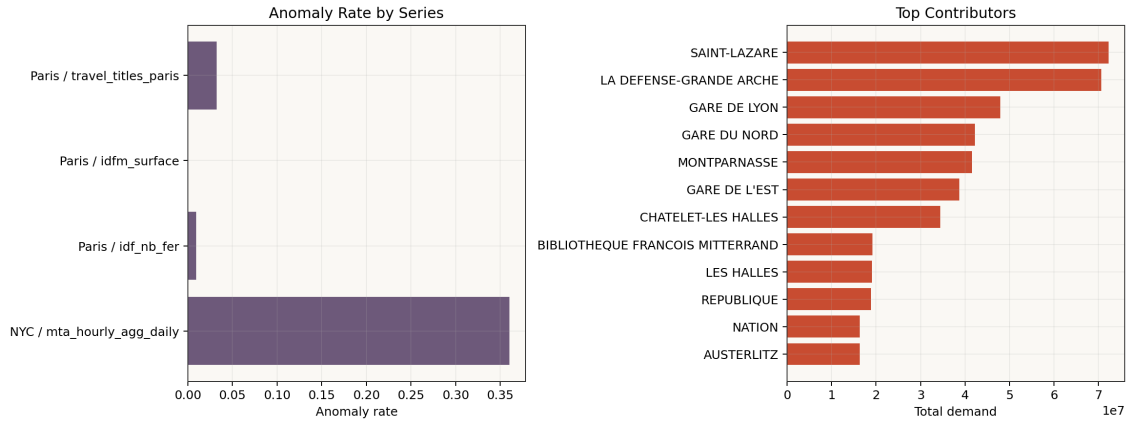


Figure 4: Anomaly-rate comparison and the highest-contributing locations.

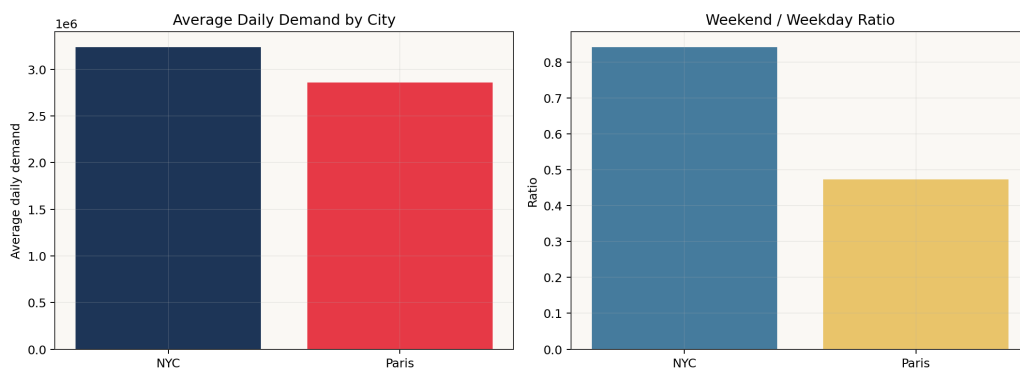


Figure 5: City-level structural comparison for memory and interpretation.

- synthetic demo weather because no real weather CSV exists in the repo

The sample mode is honest but not perfectly balanced. Paris has much longer coverage than NYC because the MTA raw file is time-ordered and even a large sample chunk only covers a small slice of dates. The full-mode pipeline is implemented for broader execution later.

Important caveat: The visuals below come from a validated sample-mode run. They teach the method correctly, but NYC should be re-run in full mode for final-grade comparison strength.

Pipeline summary

- station fact rows: 605,817
- daily rows: 962
- selected methods: 5
- weather source: `synthetic_demo`

Forecast metrics

City	MAE	RMSE	MAPE	Rows
NYC	114,313.46	135,490.96	11.3943	5
Paris	193,363.65	284,473.61	0.6625	169

Interpretation: the forecasting method is implemented and produces predictions, but the sample split is still much richer for Paris than NYC.

Weather results

- Paris in-sample weather model R^2 : 0.4023
- NYC in-sample weather model R^2 : 0.0245
- Paris precipitation correlation: 0.5503
- Paris wind correlation: 0.5756
- NYC weather correlations are weak in this sample because NYC coverage is thin.

Anomaly rates

- NYC MTA daily series anomaly rate: 0.3600
- Paris `idf_nb_fer` anomaly rate: 0.0096
- Paris `idfm_surface` anomaly rate: 0.0000
- Paris `travel_titles_paris` anomaly rate: 0.0323

Top contributors in the current run

The strongest contributors are major Paris rail hubs, led by:

- Saint-Lazare
- La Defense-Grande Arche
- Gare de Lyon
- Gare du Nord
- Montparnasse

City structure comparison

City	Avg daily value	Std daily value	Coeff. variation	Weekend/weekday ratio	Peak month
NYC	3.24e6	1.08e6	0.3325	0.8417	4
Paris	2.86e6	1.89e6	0.6623	0.4732	2

Interpretation: in the current run, Paris shows stronger relative variability and a sharper weekday orientation, while NYC appears flatter on the weekend ratio. These findings should be rechecked with a full MTA run for a stronger cross-city comparison.

7 Step-by-Step Execution Guide

1. Open the project root.
2. Use the shared virtual environment or install the dependencies listed in `requirements.txt`.
3. Run the cleaned workflow: `../.venv/bin/python scripts/run_stage_workflow.py -sample`
4. Inspect processed outputs in `data/processed/`.
5. Inspect analytical outputs in `report/results/`.
6. Read this report and the consolidated workbook.
7. If you have PostgreSQL credentials, load the outputs using `write_pipeline_outputs()` after applying the SQL schema.

8 What Was Finished

- Stage 1 analytical questions are now explicitly defined in this report and materialized in `report/results/stage1_method_plan.csv`.
- Five Stage 2 methods are implemented in code and executed locally.
- The pipeline was cleaned and made chunk-safe for large raw files.
- The report now explains the workflow, results, caveats, and next steps.

9 Remaining Caveat

The only major limitation left is data coverage balance in sample mode. The full-mode workflow is implemented, but the full MTA aggregation takes significantly longer because the raw file is 5.9 GB. That is an execution-time constraint, not a design gap in the cleaned pipeline.

A Appendix A: Basics You Need to Understand This Project

1. What is a fact table?

A fact table is the main analysis table. Each row represents one measured event or observation. In this project, the cleaned station fact table stores:

- a date;
- a region;
- a location identifier and name;

- a source;
- a metric type;
- a numeric value.

2. Why do we clean raw CSV files before modeling?

Raw files use different:

- separators (, , ; , tab);
- date formats;
- numeric conventions;
- schemas.

If we do not normalize these first, every downstream analysis becomes fragile.

3. What is a lag feature?

A lag feature is a past value used to predict the future. For example:

- `lag_1`: yesterday's demand
- `lag_7`: demand seven days ago
- `lag_14`: demand two weeks ago

These are standard forecasting features because transport demand is strongly autocorrelated.

4. What is a rolling mean?

A rolling mean averages the last n observations. It smooths noise and makes trend and baseline level easier to model. The project now computes:

- `rolling_7_mean`
- `rolling_7_std`
- `rolling_30_mean`
- `rolling_30_std`

5. What is a z-score?

A z-score tells you how far one observation is from its rolling baseline in units of standard deviation. Large absolute values are suspicious and often indicate spikes or drops.

6. Why random forest for forecasting?

Random forests are practical because they:

- work with nonlinear relationships;
- handle mixed numeric and categorical information;
- require less tuning than many advanced sequence models;
- give a strong baseline before deeper time-series models.

7. What is Isolation Forest?

Isolation Forest is an unsupervised anomaly detector. It tries to isolate observations that look unlike the rest of the data. It is useful when you do not have labels telling you which events are true anomalies.

8. What is PostgreSQL doing here?

PostgreSQL is the structured storage layer. Python prepares and enriches the data, then PostgreSQL stores it so you can query:

- daily demand;
- weather-enriched demand;
- rolling statistics;
- anomaly-oriented views and joins.

B Appendix B: Beginner-to-Finish Checklist

1. Understand the input folders: `datasets/` and `data/`.
2. Run the cleaned workflow in sample mode.
3. Open the generated CSVs in `data/processed/`.
4. Read the Stage 1 plan in `report/results/stage1_method_plan.csv`.
5. Inspect forecasting, anomaly, weather, and comparison outputs under `report/results/`.
6. Load outputs into PostgreSQL if you want SQL exploration.
7. After that, replace the synthetic weather source with a real weather file to strengthen the weather-impact method.
8. Finally, run the workflow without `-sample` when you are ready for the full raw-data pass.

C Appendix C: Important File Map

- `src/transport_analytics/pipeline.py`: chunked ingestion and canonical facts
- `src/transport_analytics/context.py`: calendar and weather enrichment
- `src/transport_analytics/features.py`: lag and rolling features
- `src/transport_analytics/methods.py`: the five Stage 2 methods
- `src/transport_analytics/postgres.py`: PostgreSQL loading
- `scripts/run_stage_workflow.py`: one-command execution
- `report/results/`: method outputs and summary tables